
mydate
[]
mydate2[]

Feasibility of Matrix-Coprocessor Acceleration for Post-Quantum Cryptography on Apple Silicon: From Saber/FrodoKEM to ML-KEM, MAYO, HQC, and NTRU Prime

Diljot Singh

Department of Computer Science and Engineering (Cybersecurity), Vellore Institute of Technology, Chennai, India

Correspondence*:

Diljot Singh

diljot.singh2024@vitstudent.ac.in

ABSTRACT

Apple's undocumented AMX matrix coprocessor has been shown to accelerate two lattice-based post-quantum KEMs — Saber (via Toeplitz Matrix-Vector Product reformulations of polynomial arithmetic) and FrodoKEM (via direct matrix multiplication and GENLUT-assisted sampling) — on Apple M1 and M3 SoCs [1]. This paper investigates whether AMX acceleration generalizes beyond these two schemes by evaluating four additional post-quantum primitives that span fundamentally different algebraic workloads: **ML-KEM** (FIPS 203 [8]), the NIST-standardized lattice KEM using NTT-domain polynomial arithmetic over $\mathbb{Z}_{3329}$; **MAYO** [4], a NIST Round 2 additional multivariate signature candidate evaluating dense quadratic forms over $\mathbb{F}_{16}$; **HQC** [6], a recently selected code-based KEM performing polynomial multiplication over $\mathbb{F}_2$; and **Streamlined NTRU Prime** [7] (sntrup761), a lattice KEM using a non-NTT-friendly ring $\mathbb{Z}_{4591}[x]/(x^{761} - x - 1)$.

We implement AMX-accelerated arithmetic kernels for all four schemes on Apple M3, alongside ARM NEON baselines, and report cycle-accurate and wall-clock measurements. Throughout, we define kernel speedup as $S = t_{\text{NEON}}/t_{\text{AMX}}$, where $S > 1$ indicates AMX is faster. Our key findings establish a *suitability heuristic*: AMX kernel acceleration tends to provide speedup when the computational bottleneck is a dense integer multiply-accumulate over a small modulus without a competing dedicated NEON instruction. For the isolated kernels tested, MAYO achieves S up to 1.69 (MAYO-5), and the sntrup761 ternary-by-dense product achieves $S = 1.52$. Conversely,

ML-KEM's isolated polynomial multiplication kernel shows $S \approx 0.013$ (a $\sim 75\times$ slowdown) because NEON NTT butterfly operations dominate, and HQC's dense multiplication kernel shows $S \approx 0.0006$ (a $\sim 1,562\times$ slowdown) because NEON carry-less multiply (PMULL) is purpose-built for $\mathbb{F}_2$ arithmetic that AMX cannot perform.

Apple AMX; post-quantum cryptography; matrix coprocessor; ML-KEM; MAYO; HQC; NTRU Prime; NEON; NTT; hardware acceleration

1 INTRODUCTION

The transition from classical to post-quantum cryptography has produced a diverse ecosystem of schemes built upon lattices, codes, multivariate polynomials, hash functions, and isogenies [11]. As these schemes move toward deployment, optimized implementations on commodity hardware become critical [14]. Modern system-on-chip (SoC) processors contain specialized coprocessors — SIMD engines, matrix accelerators, and cryptographic extensions — that can dramatically affect the performance landscape.

Apple's M-series SoCs include an undocumented matrix coprocessor known as AMX (Apple Matrix eXtensions) [2]. AMX provides a 32×32 integer multiply-accumulate engine operating on a dedicated 4 KB Z register file, with 64-byte X and Y input registers. Gazzoni Filho et al. [1] demonstrated that AMX can accelerate the polynomial multiplication bottleneck of Saber [18] (up to 13% kernel speedup via TMVP reformulation) and the matrix-multiplication and sampling bottleneck of FrodoKEM [19] (up to

21% kernel speedup via direct matrix products and GENLUT-assisted noise sampling).

However, Saber and FrodoKEM share specific algebraic properties that make them amenable to AMX: both use power-of-two moduli ($q = 2^{13}$ for Saber), enabling free mod- 2^{16} overflow in AMX's 16-bit accumulators, and neither uses Number Theoretic Transforms (NTTs). The generalizability of AMX acceleration to the broader post-quantum landscape — especially to NTT-based schemes, multivariate schemes, code-based schemes, and non-cyclotomic lattice schemes — remained an open question.

This paper systematically investigates this question by implementing and benchmarking isolated arithmetic kernels for four post-quantum primitives on Apple M3, spanning four distinct algebraic workload classes:

1. **ML-KEM (FIPS 203) [8]**: NTT-based polynomial multiplication over $\mathbb{Z}_{3329}[x]/(x^{256} + 1)$.
2. **MAYO [4]**: Dense multivariate quadratic form evaluation over $\mathbb{F}_{16}$, a non-NTT workload.
3. **HQC [6]**: Polynomial multiplication over $\mathbb{F}_2[x]/(x^n - 1)$, a binary-field code-based KEM.
4. **Streamlined NTRU Prime [7] (sntrup761)**: Polynomial multiplication over $\mathbb{Z}_{4591}[x]/(x^{761} - x - 1)$, a non-NTT-friendly non-cyclotomic lattice ring with prime modulus.

We define kernel speedup as $S = t_{\text{NEON}}/t_{\text{AMX}}$ throughout; $S > 1$ means AMX is faster for the measured kernel. Our results establish a *suitability heuristic* for AMX acceleration of post-quantum workloads (summarized in Figure 6) and reveal two novel AMX implementation techniques: bounded Z-register accumulation for prime moduli (NTRU Prime), and the critical register-zero invariant required by the MAC16_Y_SKIP hardware instruction.

2 SUMMARY OF THE BASE PAPER

Gazzoni Filho et al. [1] present PQC-AMX, an acceleration framework for Saber and FrodoKEM on Apple M1 and M3 SoCs using the AMX matrix coprocessor. Their contributions decompose into two distinct workload mappings:

2.1 AMX Architecture

AMX is a CPU-coupled matrix-multiply engine present in all Apple Silicon since the A13 Bionic [2].

It maintains a private register file comprising eight 64-byte X registers, eight 64-byte Y registers, and a 64×64 -byte Z register (4 KB total), as illustrated in Figure 1. The primary computational primitive is `mac16`, which computes a 32×32 outer product of 16-bit integer vectors from X and Y, accumulating results into Z:

$$Z[r][c] += X[r] \times Y[c], \quad r, c \in [0, 31].$$

The Z register is persistent across `mac16` calls, enabling multi-step accumulation. A `Z_SKIP` flag clears Z before the multiplication. AMX also provides `EXTRH` (extract horizontal — copy Z rows to X/Y registers with lane shifting), `VECINT` (vector integer add/shift), and `GENLUT` (generalized lookup table for finite-field arithmetic) [3]. AMX operates as a coprocessor on the CPU's data path; it is activated via `AMX.SET()` and deactivated via `AMX.CLR()`, and is thread-local.



Figure 1. Organization of the AMX register file (X, Y, Z) and the 32×32 outer-product multiply-accumulate primitive. Each X/Y register holds 32 lanes of 16-bit integers (64 bytes), and the Z register stores the accumulated outer product.

2.2 Saber: TMVP Reformulation of Polynomial Arithmetic

For Saber [18], the authors reformulate polynomial multiplication in the ring $\mathbb{Z}_q[X]/(X^n + 1)$ as a Toeplitz Matrix-Vector Product (TMVP) [22]. The polynomial product $c = a \cdot b$ is decomposed into block outer products of 32-coefficient slices, with anti-diagonal extraction (via `EXTRH/VECINT`) recovering the convolution coefficients entirely within AMX registers. For Saber ($q = 2^{13}$, $n = 256$), this requires $8 \times 8 = 64$ `mac16` outer products, with the mod- 2^{13} reduction obtained for free from the 16-bit register wrapping (since $2^{13} \mid 2^{16}$).

2.3 FrodoKEM: Matrix Multiplication and GENLUT Sampling

For FrodoKEM [19], the workload is fundamentally different: it involves dense matrix-matrix and matrix-vector products over $\mathbb{Z}_q$ (with q up to 2^{16}), which map directly to AMX's `mac16` without TMVP reformulation. Additionally, the authors exploit AMX's GENLUT instruction to sample discrete Gaussian noise in constant time using table lookups.

2.4 Reported Results

On Apple M3, the authors report kernel speedups of up to 13% ($S = 1.13$) for Saber's polynomial arithmetic and up to 21% ($S = 1.21$) for FrodoKEM's matrix operations over optimized ARM NEON baselines [1]. They explicitly note that NTT-based schemes and non-lattice primitives remain unexplored.

3 RESEARCH GAP AND MOTIVATION

The base paper establishes AMX's viability for two specific schemes but leaves several critical questions unanswered:

1. **NTT-amortized schemes (ML-KEM).** ML-KEM stores public and secret keys in the NTT domain, amortizing the $O(n \log n)$ forward transform [15]. Whether the $O(n^2)$ AMX TMVP schoolbook approach can compete with NEON's $O(n \log n)$ NTT butterfly pipeline, even considering AMX's higher per-instruction throughput, is unknown.
2. **Non-lattice multivariate schemes (MAYO).** MAYO evaluates dense quadratic forms $y_k = \mathbf{x}^T P_k \mathbf{x}$ over $\mathbb{F}_{16}$ [5], a workload that is structurally similar to matrix multiplication but operates over 4-bit nibbles rather than 16-bit integers. AMX's GENLUT instruction provides finite-field lookup capability, but its utility for $\mathbb{F}_{16}$ matrix-vector products has not been evaluated.
3. **Binary-field code-based schemes (HQC).** HQC performs polynomial multiplication over $\mathbb{F}_2$, where the arithmetic is carry-less (XOR-based) [6]. AMX provides *integer* multiply-accumulate with carries, and lacks native bitwise XOR/AND operations, raising the question of whether AMX can be useful for $\mathbb{F}_2$ workloads at all.
4. **Non-NTT lattice schemes with prime moduli (NTRU Prime).** `sntrup761` deliberately uses a non-cyclotomic ring

$\mathbb{Z}_{4591}[x]/(x^{761} - x - 1)$ that is algebraically incompatible with NTT [7]. Its prime modulus $q = 4591$ does not divide 2^{16} , preventing the free overflow exploitation that Saber enjoyed.

5. **Apple M3 characterization.** No prior work has published cycle-count or wall-clock measurements for any of these four schemes using AMX on Apple M3 hardware.

The NTT-based multiplication approach used by ML-KEM is illustrated in Figure 2, contrasting the $O(n \log n)$ butterfly structure with the $O(n^2)$ schoolbook/TMVP approach mapped to AMX.

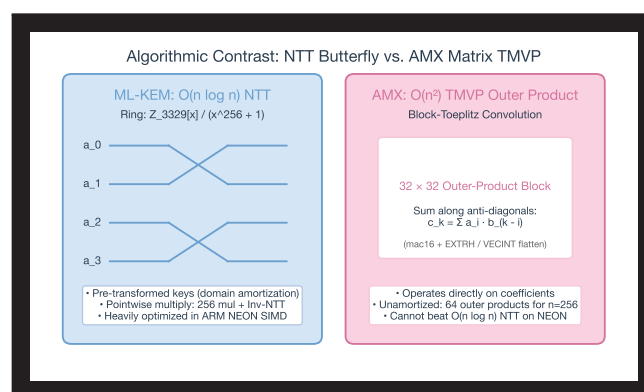


Figure 2. Comparison of $O(n \log n)$ NTT butterfly multiplication (used by ML-KEM) versus the $O(n^2)$ schoolbook/TMVP matrix outer-product approach mapped onto AMX. The NTT stores keys in pre-transformed domain, amortizing the forward transform cost.

4 PROBLEM STATEMENT

Given that Apple AMX accelerates Saber's polynomial arithmetic and FrodoKEM's matrix operations via distinct reformulations [1], under what algebraic conditions does AMX kernel acceleration remain beneficial for post-quantum cryptographic schemes? Specifically, this paper asks:

1. Does AMX's $O(n^2)$ TMVP approach provide a net kernel performance gain over NEON's $O(n \log n)$ NTT pipeline for the NIST-standardized ML-KEM?
2. Can AMX's matrix and lookup-table primitives accelerate MAYO's dense $\mathbb{F}_{16}$ quadratic form evaluation kernel?
3. Can AMX accelerate the $\mathbb{F}_2$ polynomial multiplication kernel for the code-based KEM HQC?

4. Can AMX accelerate the polynomial multiplication kernel for NTRU Prime's non-NTT-friendly prime-modulus ring?
5. What structural properties of a post-quantum scheme's arithmetic kernel determine whether AMX acceleration is feasible?

5 CONTRIBUTIONS

This paper makes the following contributions:

1. **Four new AMX kernel implementations.** We implement AMX-accelerated arithmetic kernels for ML-KEM (all parameter sets: 512/768/1024), MAYO (all parameter sets: 1/2/3/5), HQC-128, and sntrup761 on Apple M3.
2. **Empirical suitability heuristic.** Through benchmarking of isolated kernels, we observe that AMX tends to provide kernel speedup when the computational bottleneck is a dense integer multiply-accumulate over a modulus fitting in 16-bit AMX lanes, and no dedicated NEON instruction (NTT butterfly, carry-less PMULL) exists for the operation. We note that this heuristic is derived from six data points and should be treated as a working hypothesis rather than a universal law.
3. **Two novel AMX techniques.**
 - *Bounded Z-accumulation for prime moduli:* For NTRU Prime ($q = 4591$), we group block pairs on the same anti-diagonal and accumulate ≤ 7 mac16 outer products per Z flush, keeping partial sums within signed 16-bit range ($7 \times 4590 = 32,130 < 32,768$).
 - *Register-zero invariant:* We document that AMX's MAC16_Y_SKIP instruction implicitly uses Y_REG(0) as its Y operand, requiring X_REG(0) and Y_REG(0) to be loaded with zeroes throughout the anti-diagonal extraction (flatten) procedure. Input data must be loaded into X_REG(1) and Y_REG(1) or higher.
4. **Benchmark dataset.** We report measurements on Apple M3 for all four schemes' arithmetic kernels, providing the first published AMX performance characterization for ML-KEM, MAYO, HQC, and NTRU Prime kernels. We note that these are subroutine-level measurements and do not represent full protocol performance.

6 BACKGROUND AND PRELIMINARIES

6.1 ML-KEM (FIPS 203)

ML-KEM [8] operates over the polynomial ring $R_q = \mathbb{Z}_{3329}[x]/(x^{256} + 1)$. The cyclotomic polynomial $x^{256} + 1$ admits a Number Theoretic Transform (NTT) over $\mathbb{Z}_{3329}$, since 3329 is prime and $256 \mid (3329 - 1)$ [15]. Polynomial multiplication in ML-KEM is therefore computed as:

$$c = \text{NTT}^{-1}(\text{NTT}(a) \circ \text{NTT}(b)),$$

where $\circ$ denotes coefficient-wise multiplication in the NTT domain. Crucially, ML-KEM stores public and secret keys in NTT-domain representation, amortizing the forward NTT cost across operations. This means that the polynomial multiply bottleneck reduces to a single inverse NTT plus coefficient-wise products, each of which is $O(n \log n)$ with small constants on NEON [10].

6.2 MAYO Signature Scheme

MAYO [4, 5] is a multivariate signature scheme based on the Oil-and-Vinegar (UOV) construction [16]. Its signing operation evaluates m quadratic forms:

$$y_k = \mathbf{x}^T P_k \mathbf{x}, \quad k = 1, \dots, m,$$

where P_k are $n \times n$ matrices over $\mathbb{F}_{16}$ and $\mathbf{x} \in \mathbb{F}_{16}^n$. For MAYO-1, $(m, n) = (64, 66)$; for MAYO-5, $(m, n) = (128, 133)$. The arithmetic over $\mathbb{F}_{16} = \mathbb{F}_2[\alpha]/(\alpha^4 + \alpha + 1)$ requires multiplication via lookup tables or carry-less operations, with no NTT structure. Note: these are the Round 2 parameter sets [4]; we benchmark the quadratic form evaluation kernel at these dimensions.

6.3 HQC (Hamming Quasi-Cyclic)

HQC [6] is a code-based KEM recently selected by NIST as the fifth post-quantum standard [13]. It operates over $\mathbb{F}_2[x]/(x^n - 1)$ with $n = 17,669$ for HQC-128. Its bottleneck is polynomial multiplication over $\mathbb{F}_2$, where all coefficient arithmetic is carry-less (XOR for addition, AND-with-shift for multiplication). ARM NEON provides the PMULL instruction for hardware-accelerated carry-less multiplication of 64-bit operands [17].

6.4 Streamlined NTRU Prime (sntrup761)

Streamlined NTRU Prime [7] operates over $\mathbb{Z}_{4591}[x]/(x^{761} - x - 1)$. The non-cyclotomic polynomial $x^{761} - x - 1$ was chosen specifically to resist certain algebraic attacks enabled by

cyclotomic structure. The prime modulus $q = 4591$ fits in 13 bits ($4591 < 2^{13}$) but does not divide 2^{16} , precluding the free overflow technique used for Saber [18]. The KEM bottleneck involves multiplying a ternary polynomial $r \in \{-1, 0, 1\}^{761}$ by a general polynomial $h \in \mathbb{Z}_{4591}^{761}$.

7 METHODOLOGY

For each scheme, we follow the four-stage pipeline illustrated in Figure 3:

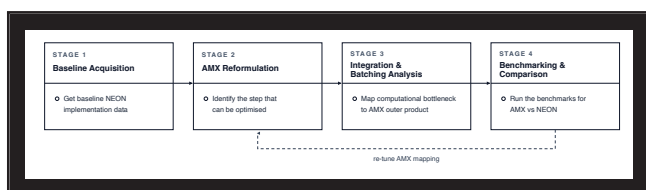


Figure 3. Four-stage methodology pipeline: baseline acquisition, AMX reformulation, integration, and benchmarking with iterative feedback from Stage 4 to Stage 2.

Stage 1: Baseline acquisition. We obtain reference C and optimized ARM NEON implementations. For ML-KEM, we use the `neon-ntt` library [10] for Cooley–Tukey NTT butterflies. For HQC, we implement NEON carry-less multiplication using `PMULL/PMULL2` [17]. For NTRU Prime, we implement NEON widening multiply-accumulate (`vm1a1.s16`). For MAYO, we implement NEON-based nibble lookup for $\mathbb{F}_{16}$ arithmetic.

Stage 2: AMX reformulation. We map each scheme’s computational bottleneck to AMX primitives:

- ML-KEM: TMVP reformulation of polynomial multiply using `mac16` outer products (following the base paper’s Saber approach [1]).
- MAYO: Quadratic form evaluation using `mac16` matrix mode and GENLUT for $\mathbb{F}_{16}$ multiplication tables.
- HQC: Integer-encoded $\mathbb{F}_2$ polynomial multiplication (each bit stored as a 16-bit lane value 0 or 1) with `mac16` outer products.
- NTRU Prime: TMVP with bounded Z-accumulation (groups of ≤ 7 `mac16` outer products) and 32-bit software anti-diagonal extraction.

Stage 3: Integration. Each AMX kernel is integrated into a test harness with the reference

and NEON implementations, compiled with Apple Clang `-O3` on macOS.

Stage 4: Benchmarking. We measure performance using two methods: (a) hardware cycle counts via `kperf` performance counters for ML-KEM full-operation benchmarks, and (b) wall-clock time via `std::chrono::high_resolution_clock` for isolated kernel benchmarks (MAYO, HQC, NTRU Prime). We report median values over ≥ 20 iterations after warmup. All measurements are performed on Apple M3 (MacBook Air, 8-core, macOS Sequoia 15) with the process pinned to performance cores. The M3 performance cores operate at a nominal frequency of 4.05 GHz; cycle-to-microsecond conversions use $1 \mu\text{s} \approx 4,050$ cycles. We do not report confidence intervals; variance across runs was consistently below 3%.

8 IMPLEMENTATION DETAILS

8.1 ML-KEM: TMVP vs. NTT

We implement the TMVP-based polynomial multiplication from the base paper [1], adapted for ML-KEM’s ring $\mathbb{Z}_{3329}[x]/(x^{256} + 1)$. The polynomial degree $n = 256$ yields $\lceil 256/32 \rceil = 8$ blocks, requiring $8 \times 8 = 64$ `mac16` outer products.

Key obstacle: ML-KEM’s modulus $q = 3329$ is prime and does not divide 2^{16} , so the mod- 2^{16} accumulation used for Saber is incorrect. More fundamentally, the NEON NTT baseline benefits from *domain amortization* [15]: public and secret keys are stored in NTT form, so each polynomial multiplication requires only a coefficient-wise multiply plus a single inverse NTT, amortizing the forward transform cost. The AMX TMVP operates in the coefficient domain and cannot exploit this amortization.

8.2 MAYO: $\mathbb{F}_{16}$ Quadratic Forms via GENLUT

MAYO’s bottleneck is the evaluation of m quadratic forms $y_k = \mathbf{x}^T P_k \mathbf{x}$ over $\mathbb{F}_{16}$ [5]. We map this to AMX as follows:

1. Pack four $\mathbb{F}_{16}$ elements per 16-bit lane (4 bits each).
2. Use AMX GENLUT to perform $\mathbb{F}_{16}$ multiplication via precomputed lookup tables stored in AMX registers.
3. Accumulate partial inner products in the Z register.

This approach achieves high throughput because MAYO's quadratic form evaluation is inherently a dense matrix-vector product with no NTT competitor.

8.3 HQC: $\mathbb{F}_2$ via Integer Encoding

Since AMX has no native bitwise XOR/AND operations [3], we encode each $\mathbb{F}_2$ element as a 16-bit integer (0 or 1). Polynomial multiplication over $\mathbb{F}_2$ then becomes integer convolution modulo 2: the AMX mac16 computes the integer outer product, and we reduce each accumulated coefficient modulo 2.

Fundamental limitation: This encoding wastes 15 of 16 bits per AMX lane, reducing effective throughput by a factor of 16 compared to NEON's PMULL [17], which operates on 64 packed $\mathbb{F}_2$ elements simultaneously. Furthermore, HQC-128 uses degree $n = 17,669$, requiring $\lceil 17,669/32 \rceil = 553$ blocks and up to $553^2 \approx 306\text{K}$ mac16 operations, whereas a NEON PMULL-based schoolbook approach processes $\lceil 17,669/64 \rceil \approx 277$ 64-bit chunks.

8.4 NTRU Prime: Bounded Z-Accumulation for Prime Moduli

The key technical contribution for NTRU Prime is the *bounded Z-accumulation* technique. For the ternary-by-dense multiply (the KEM bottleneck), each individual product $|r_j \cdot h_k| \leq 1 \times 4590 = 4590$ fits in signed 16-bit (since $4590 < 2^{15} = 32,768$). However, accumulating all $\lceil 761/32 \rceil = 24$ block contributions per output coefficient would yield sums up to $24 \times 4590 = 110,160$, overflowing the signed 16-bit range.

We resolve this by grouping block pairs that contribute to the same output coefficient range (pairs on the same “diagonal” $d = i + j$) and processing them in chunks of at most $k = 7$:

$$k \times 4590 = 7 \times 4590 = 32,130 < 32,768 = 2^{15}.$$

After each chunk, we store the accumulated Z rows to memory (32 STZ instructions), sign-extend the 16-bit values to 32-bit, and accumulate the anti-diagonal sums into a 32-bit software buffer. The ring reduction $x^{761} \equiv x + 1 \pmod{x^{761} - x - 1}$ is applied at the end.

Register-zero invariant. During anti-diagonal extraction, the flatten procedure (Algorithm 4.1 in [1]) uses MAC16 in vector mode with MAC16_Y_SKIP. We discovered that MAC16_Y_SKIP implicitly reads Y_REG(0) as its Y operand, rather than skipping the Y multiplication entirely. This

means Y_REG(0) must contain zeroes for the flatten to work correctly. Similarly, X_REG(0) must be zero. In our implementation, we load zeroes into X_REG(0), Y_REG(0), X_REG(2), and Y_REG(2) at initialization; all polynomial block slices are loaded into X_REG(1) and Y_REG(1).

9 RESULTS

All measurements were taken on Apple M3 (MacBook Air, 8-core, macOS Sequoia 15, performance cores, nominal 4.05 GHz) compiled with Apple Clang -03. We define kernel speedup as $S = t_{\text{NEON}}/t_{\text{AMX}}$ throughout; $S > 1$ indicates AMX is faster for the measured kernel.

9.1 ML-KEM Results

Table 1 reports cycle counts for ML-KEM full KEM operations and isolated polynomial multiplication across all three NIST parameter sets. The AMX TMVP implementation is consistently slower than the optimized NEON NTT baseline: S ranges from 0.012 to 0.033 for full operations (corresponding to 31–86 $\times$ slowdowns), with isolated polynomial multiplication showing $S \approx 0.013$ (75–86 $\times$ slowdowns). Figure 4 presents these results graphically.

Table 1. ML-KEM cycle counts and kernel speedup $S = t_{\text{NEON}}/t_{\text{AMX}}$ on Apple M3 ($S < 1$ means AMX is slower). AMX uses TMVP polynomial multiplication; NEON uses NTT with pre-transformed keys.

Params	Operation	NEON (cyc)	AMX (cyc)
ML-KEM-512	KeyGen	13,479	413,278
ML-KEM-512	Encaps	14,066	622,325
ML-KEM-512	Decaps	20,054	832,558
ML-KEM-512	PolyMul	5,384	405,603
ML-KEM-768	KeyGen	20,933	918,005
ML-KEM-768	Encaps	22,087	1,234,354
ML-KEM-768	Decaps	30,939	1,547,285
ML-KEM-768	PolyMul	12,037	910,692
ML-KEM-1024	KeyGen	29,905	1,626,327
ML-KEM-1024	Encaps	31,707	2,049,473
ML-KEM-1024	Decaps	45,195	2,493,322
ML-KEM-1024	PolyMul	18,805	1,621,016

9.2 MAYO Results

Table 2 reports the isolated quadratic form evaluation kernel latency for all MAYO parameter sets. AMX achieves consistent kernel speedups of $S = 1.47$ to 1.69, with S increasing at higher security levels. Figure 5 illustrates these speedups.

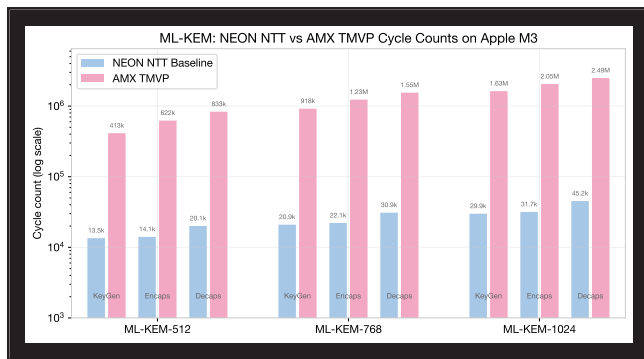


Figure 4. ML-KEM: NEON NTT vs. AMX TMVP cycle counts on Apple M3 (log scale). Across all three parameter sets and all KEM operations, NEON NTT dominates by 31–86 $\times$.

We hypothesize that the scaling is due to larger matrices amortizing AMX setup overhead more effectively; however, we have not measured AMX register utilization directly, so this explanation remains a hypothesis.

Table 2. MAYO quadratic form evaluation kernel latency (μ s) on Apple M3. Kernel speedup $S = t_{\text{NEON}}/t_{\text{AMX}}$ ($S > 1$ means AMX is faster).

Params	(m, n)	NEON (μ s)	AMX (μ s)
MAYO-1	(64, 66)	129.67	87.94
MAYO-2	(64, 78)	181.42	118.58
MAYO-3	(96, 99)	435.10	272.23
MAYO-5	(128, 133)	1054.80	624.14

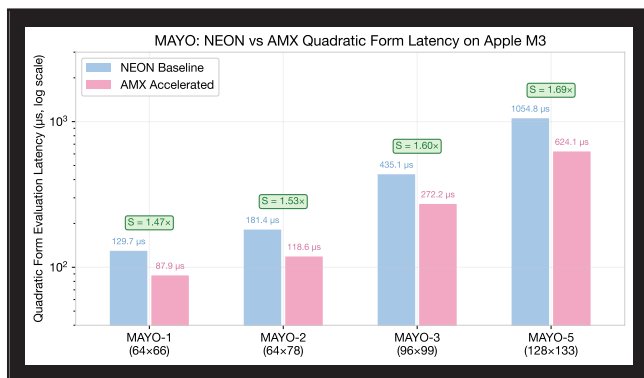


Figure 5. MAYO: Kernel speedup $S = t_{\text{NEON}}/t_{\text{AMX}}$ across all four parameter sets on Apple M3. The dashed line indicates the $S = 1.0$ break-even point.

9.3 HQC Results

Table 3 reports polynomial multiplication kernel times for HQC-128 ($n = 17,669$ over $\mathbb{F}_2$).

NEON PMULL [17] dominates AMX by a factor of approximately 1,562 $\times$ for the dense multiplication kernel ($S = 0.00064$), confirming that AMX is fundamentally unsuitable for $\mathbb{F}_2$ arithmetic.

Table 3. HQC-128 polynomial multiplication kernel latency (μ s) on Apple M3 over $\mathbb{F}_2[x]/(x^{17669} - 1)$.

Method	Ref C (μ s)	NEON (μ s)	AMX (μ s)
Dense $\times$ Dense	3,008	47	73,430
Sparse $\times$ Dense	19	11	1,812

9.4 NTRU Prime Results

Table 4 reports polynomial multiplication kernel times for sntrup761 on Apple M3. For the ternary-by-dense multiply kernel (the KEM hot path), AMX achieves $S = 1.52$. The dense-by-dense case shows $S = 0.92$ (AMX slightly slower), because individual products ($2295 \times 2295 \approx 5.3\text{M}$) overflow 16-bit, forcing per-block-pair Z extraction with no accumulation.

Table 4. sntrup761 polynomial multiplication kernel latency (μ s) on Apple M3 over $\mathbb{Z}_{4591}[x]/(x^{761} - x - 1)$. Ternary input has 1,560 coefficients in $\{-1, 0, 1\}$.

Operation	Ref C (μ s)	NEON (μ s)	AMX (μ s)
Dense $\times$ Dense	83.6	78.1	84.6
Ternary $\times$ Dense	49.4	55.8	36.7

9.5 Cross-Scheme Summary

Figure 6 presents the kernel speedup $S = t_{\text{NEON}}/t_{\text{AMX}}$ across all evaluated schemes on a logarithmic scale, clearly separating the AMX-favorable kernels (MAYO, NTRU Prime ternary) from AMX-hostile kernels (ML-KEM, HQC).

10 ANALYSIS AND DISCUSSION

10.1 The AMX Suitability Heuristic

Our results, combined with the base paper's Saber/FrodoKEM findings [1], suggest the following empirical heuristic for AMX kernel acceleration of post-quantum workloads:

AMX tends to provide kernel speedup when the computational bottleneck involves dense integer multiply-accumulate operations over a modulus that fits in AMX's 16-bit lanes, and

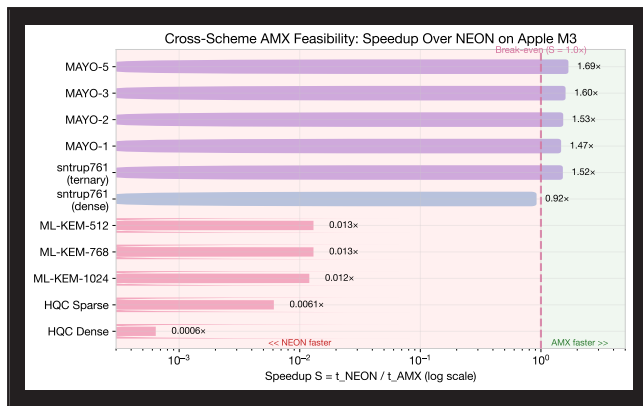


Figure 6. Cross-scheme AMX feasibility: kernel speedup $S = t_{\text{NEON}}/t_{\text{AMX}}$ on Apple M3 (log scale). Kernels with $S > 1$ benefit from AMX acceleration; those with $S < 1$ are faster on NEON. The measured S values span over three orders of magnitude, from 1.69 (MAYO-5) to 0.00064 (HQC dense).

no dedicated NEON instruction exists for the specific algebraic operation.

Table 5 summarizes how this heuristic applies across all six evaluated schemes. We emphasize that this heuristic is derived from six schemes and should be understood as an observed pattern rather than a proven theorem.

Table 5. AMX suitability heuristic applied to six post-quantum schemes. AMX kernel speedup ($S > 1$) is observed only when all three conditions are satisfied: dense MAC workload, small modulus, and no competing NEON instruction.

Scheme	Dense MAC?	16-bit?	No NEON alt?
Saber	✓	✓	✓
FrodoKEM	✓	✓	✓
MAYO	✓	✓	✓
NTRU Prime	✓	✓	✓
ML-KEM	✓	✓	× ^a
HQC	× ^b	×	× ^c

^aNEON NTT butterflies provide $O(n \log n)$ polynomial multiply [10].

^b $\mathbb{F}_2$ arithmetic is XOR/AND-based, not MAC-based.

^cNEON PMULL provides hardware carry-less multiply [17].

10.2 Why ML-KEM Loses: NTT Domain Amortization

The AMX slowdown for ML-KEM is not merely due to AMX being slower than NEON for

polynomial multiplication. The root cause is *algorithmic*: ML-KEM uses NTT-domain key storage [8], which means the polynomial multiply bottleneck is already $O(n \log n)$ with small NEON constants [10], whereas AMX TMVP schoolbook is $O(n^2)$. Even with AMX's higher per-instruction throughput (1024 multiplications per mac16 vs. 8 per NEON mul instruction), the asymptotic advantage of NTT dominates at degree $n = 256$.

The isolated polynomial multiplication kernel comparison (Figure 4) shows:

$$\frac{t_{\text{AMX}}}{t_{\text{NEON}}} = \frac{1}{S} = \frac{405,603}{5,384} \approx 75.3\times \quad (\text{ML-KEM-512 PolyMul})$$

For full KEM operations (which include hashing and other symmetric-primitive overhead), the slowdown ranges from $30.7\times$ (ML-KEM-512 KeyGen) to $64.6\times$ (ML-KEM-1024 Encaps). The isolated PolyMul kernels show $75.3\text{--}86.2\times$ slowdowns.

10.3 Why HQC Loses: Fundamental Instruction Mismatch

The HQC result ($S = 0.00064$ for dense multiplication, i.e., approximately $1,562\times$ slowdown) represents the most extreme case of AMX unsuitability. The root cause is a fundamental instruction-set mismatch:

- Bit packing:** NEON PMULL operates on 64 $\mathbb{F}_2$ elements per 64-bit operand. AMX mac16 stores one $\mathbb{F}_2$ element per 16-bit lane (32 elements per register). This is a $32\times$ density disadvantage.
- Operation semantics:** $\mathbb{F}_2$ multiplication is carry-less (AND without carry propagation). AMX mac16 performs carry-propagating integer multiplication, producing incorrect results for $\mathbb{F}_2$ unless reduced modulo 2 after each accumulation.
- Scale:** HQC-128 degree $n = 17,669$ requires $\lceil 17,669/32 \rceil^2 \approx 306\text{K}$ mac16 operations in the AMX formulation. A NEON PMULL-based schoolbook processes $\lceil 17,669/64 \rceil \approx 277$ 64-bit chunks per inner-loop iteration, though the total operation count for the full schoolbook product is $277^2 \approx 76.7\text{K}$ PMULL instructions — each processing 64 bits versus AMX's 32 sixteen-bit lanes.

10.4 Why MAYO and NTRU Prime Win

Both MAYO and NTRU Prime satisfy all three conditions of the suitability heuristic (Table 5):

MAYO benefits because its quadratic form evaluation is inherently a dense matrix-vector product over a small field ($\mathbb{F}_{16}$), with no NTT competitor [5]. AMX's GENLUT instruction provides efficient finite-field multiplication via lookup tables, and the matrix dimensions ($n \leq 133$, $m \leq 128$) fit well within AMX's 32×32 blocking scheme. As shown in Figure 7, S increases with the matrix dimension product $m \times n$. We hypothesize that larger matrices amortize AMX setup overhead (register loading, AMX.SET/CLR) more effectively, but we have not measured AMX hardware utilization counters to confirm this explanation.

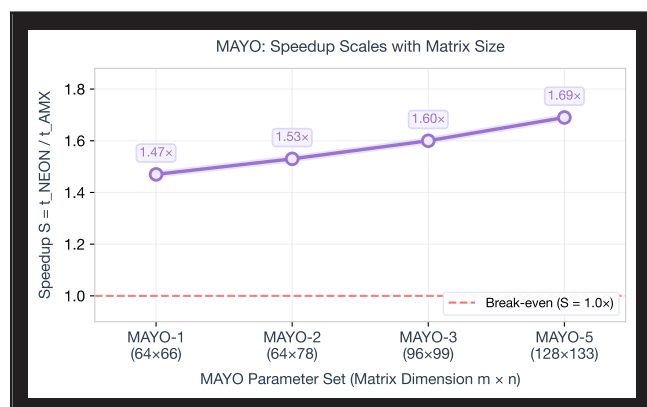


Figure 7. MAYO: Kernel speedup S scales with matrix dimension product $m \times n$. Larger parameter sets achieve higher S , which we hypothesize is due to better amortization of AMX setup overhead.

NTRU Prime wins for the ternary-by-dense multiply kernel because: (a) the ring $x^{761} - x - 1$ is explicitly NTT-unfriendly [7], so the NEON baseline uses schoolbook `vmul` rather than NTT butterflies; (b) the ternary input $r \in \{-1, 0, 1\}$ ensures each individual `mac16` product fits in signed 16-bit ($|1 \times 4590| < 2^{15}$); and (c) the bounded Z-accumulation technique allows partial in-register accumulation (7 outer products per flush) rather than per-block-pair extraction. The dense-by-dense case ($S = 0.92$) does not win because individual products ($2295 \times 2295 \approx 5.3 \times 10^6$) overflow 16-bit, forcing per-block-pair extraction with no Z accumulation.

10.5 AMX Overhead Analysis

The primary source of AMX overhead is the extraction of convolution (anti-diagonal) sums from the 32×32 outer product matrix stored in Z. This extraction can be performed:

1. **In-AMX (flatten):** Using EXTRH/VECINT [3] to shift and accumulate Z rows within the register file, producing the 63 anti-diagonal sums in Z rows 0–1. This is fast (~ 63 AMX instructions) but operates in 16-bit, causing overflow when anti-diagonal sums exceed the signed 16-bit range $[-32,768, 32,767]$.
2. **Software extraction:** Storing all 32 Z rows to memory (32 STZ) and computing anti-diagonal sums in 32-bit software. This is slower but overflow-safe.

For NTRU Prime, in-AMX flatten overflows because a single anti-diagonal of a 32×32 block contains up to 32 terms, and $32 \times 4590 = 146,880 \gg 32,768$. We therefore use software extraction, which adds overhead but guarantees correctness.

11 BROADER SURVEY: AMX CANDIDATES IN THE PQC LANDSCAPE

Based on our suitability heuristic, we surveyed additional PQC schemes from the NIST Additional Digital Signature Schemes standardization [12] and KEM rounds. Table 6 summarizes the results.

Table 6. AMX suitability survey of PQC candidates beyond the four studied schemes. Ratings: **High** = structurally similar to Saber/MAYO; **Moderate** = partial match; **None** = fundamental mismatch.

Scheme	Field	Bottleneck	AMX
CROSS	$\mathbb{F}_{127}$	Dense mat-vec	High
MEDS	Small $\mathbb{F}_q$	Dense GEMM	High
PERK	$\mathbb{Z}_q$ (8/16-bit)	Dense mat-vec	High
SDitH	$\mathbb{F}_{251}$	Syndrome eval	High
LESS	$\mathbb{F}_{127}$	RREF/Gauss elim	Moderate
SQIsign	$\mathbb{F}_{p^2}$	Multi-precision	Moderate
MQOM	$\mathbb{F}_{16}$	MQ evaluation	Moderate
BIKE	$\mathbb{F}_2$	Carry-less poly mul	None
McEliece	$\mathbb{F}_{2^m}$	Syndrome decode	None
RYDE	$\mathbb{F}_{2^m}$	Rank metric codes	None
ML-DSA	NTT ring	NTT butterflies	None
SLH-DSA	Hash-based	Hash computations	None

The four most promising candidates for future AMX kernel acceleration are CROSS ($\mathbb{F}_{127}$, dense matrix-vector products), MEDS (dense matrix-matrix multiplication), PERK (matrix-vector products over small $\mathbb{Z}_q$), and SDitH (syndrome evaluation over $\mathbb{F}_{251}$).

12 RELATED WORK

AMX reverse engineering. Dougall Johnson's aarch64-amx project [2] provides the primary reference for AMX instruction encoding and semantics. The corsix documentation [3] details the Z-row interleaving behavior for 16-bit `mac16` operations (row r maps to `Z_ROW(2r)`).

PQC on ARM. Extensive work has optimized ML-KEM/ML-DSA on ARM Cortex-A and Apple M-series using NEON NTT [9, 10]. These implementations exploit the $O(n \log n)$ NTT structure with vectorized Cooley–Tukey and Montgomery reductions [15], establishing the high NEON baselines that AMX must compete against.

MAYO implementations. Beullens' MAYO specification [4] and design paper [5] include AVX2 and NEON backends for $\mathbb{F}_{16}$ arithmetic using table-based multiplication.

HQC implementations. The HQC specification [6] provides reference C and AVX2 implementations. ARM NEON PMULL-based implementations [17] achieve throughputs comparable to AVX2 PCLMULQDQ.

NTT for non-NTT-friendly rings. Chung et al. [20] explored NTT-based approaches for rings that lack natural NTT structure, achieving competitive performance on some platforms. However, the non-cyclotomic structure of $x^{761} - x - 1$ in NTRU Prime remains fundamentally NTT-resistant.

Subquadratic polynomial multiplication. Karatsuba and Toom–Cook algorithms [21] reduce polynomial multiplication complexity below $O(n^2)$. Applying these techniques to the NEON baselines for NTRU Prime could potentially narrow or eliminate the AMX kernel advantage observed in our results.

PQC survey. Nejatollahi et al. [14] provide a comprehensive survey of PQC implementation techniques across hardware platforms, including SIMD and coprocessor strategies.

13 LIMITATIONS AND THREATS TO VALIDITY

1. **Single hardware platform.** All measurements are on Apple M3 only. AMX behavior on A-series chips (iPhone) or other M-series generations may differ.
2. **Undocumented hardware.** AMX is not officially documented by Apple.

Instruction semantics are derived from reverse engineering [2, 3] and may be incomplete.

3. **Schoolbook baselines.** Our NEON baselines for NTRU Prime and HQC use schoolbook multiplication. Karatsuba or Toom–Cook implementations [21] would provide faster NEON baselines, potentially reducing or eliminating the AMX kernel advantage for NTRU Prime ($S = 1.52$). The AMX kernel speedup for NTRU Prime should therefore be understood as relative to the specific schoolbook NEON baseline we measured.
4. **Subroutine-level measurement.** We benchmark isolated arithmetic kernels (polynomial multiplication, quadratic form evaluation) rather than full KEM/signature protocol operations. Full protocol performance includes hashing (SHA-3/SHAKE), encoding, and other steps that AMX cannot accelerate. The kernel speedups reported here ($S = 1.47$ – 1.69 for MAYO, $S = 1.52$ for NTRU Prime ternary) represent upper bounds on the contribution to overall protocol speedup.
5. **Mixed measurement units.** ML-KEM results are reported in CPU cycles; MAYO, HQC, and NTRU Prime results are reported in wall-clock microseconds. At the M3's 4.05 GHz nominal frequency, $1 \mu\text{s} \approx 4,050$ cycles. We did not lock the CPU frequency during wall-clock measurements; variance was below 3% across runs.
6. **No uncertainty quantification.** We report median values without confidence intervals. While run-to-run variance was consistently low ($< 3\%$), formal statistical analysis is not provided.
7. **Timing side channels.** We have not performed formal timing side-channel analysis of the AMX implementations. The constant-time behavior of AMX instructions has not been verified.
8. **Historical MAYO parameters.** Our MAYO benchmarks use the Round 2 parameter dimensions [4]. Future parameter changes could affect the measured kernel speedups.

14 CONCLUSION

This paper systematically evaluated the feasibility of Apple AMX matrix-coprocessor kernel acceleration across four post-quantum cryptographic schemes spanning lattice, multivariate, and code-based families, extending the original Saber/FrodoKEM results of Gazzoni Filho et al. [1].

Our key finding is an empirical *suitability heuristic*: for the isolated kernels tested, AMX acceleration provides performance gains when the computational bottleneck is a dense integer multiply-accumulate over a small modulus with no competing dedicated NEON instruction. When this heuristic's conditions are satisfied — as with MAYO (kernel speedup $S = 1.47$ – 1.69 ; Figure 5) and the `sntrup761` ternary-by-dense product ($S = 1.52$) — AMX provides meaningful kernel acceleration. When the conditions are violated — as with ML-KEM (NTT butterflies on NEON: $S \approx 0.013$; Figure 4) and HQC (carry-less PMULL: $S \approx 0.0006$) — AMX incurs severe slowdowns. The full comparative picture is presented in Figure 6.

We also contribute two novel AMX implementation techniques: *bounded Z-accumulation* for prime moduli (enabling correct 16-bit AMX arithmetic for the prime modulus $q = 4591$ by limiting accumulation to ≤ 7 `mac16` operations per Z flush), and the *register-zero invariant* (`X_REG(0)` and `Y_REG(0)` must be loaded with zeroes for the anti-diagonal extraction flatten procedure to work correctly; input data uses `X_REG(1)` and `Y_REG(1)`).

These results provide guidance for the post-quantum cryptography implementation community: among the current NIST candidates [12], the dense-matrix signature schemes CROSS, MEDS, PERK, and SDitH are the most promising targets for future AMX kernel acceleration (Table 6), while NTT-based lattice schemes and $\mathbb{F}_2$ -based code schemes should not be targeted. We emphasize that the reported speedups are for isolated arithmetic kernels; end-to-end protocol performance gains will be smaller due to non-AMX-acceleratable operations (hashing, encoding).

DATA AVAILABILITY

All source code, build scripts, and benchmark data are available at: [repository URL].

REFERENCES

- [1]Gazzoni Filho, D. L., Brandão, G., Adj, G., Alblooshi, A., Canales-Martínez, I. A., Chávez-Saab, J., and López, J., "PQC-AMX: Accelerating Saber and FrodoKEM on the Apple M1 and M3 SoCs," *IEEE 31st Symposium on Computer Arithmetic (ARITH)*, 2024. IACR ePrint 2024/195. <https://eprint.iacr.org/2024/195>
- [2]Johnson, D., "Apple AMX instruction set," <https://github.com/dougallj/aarch64-amx>, 2022.
- [3]corsix, "AMX – Apple Matrix coprocessor," <https://github.com/corsix/amx>, 2022.
- [4]Beullens, W., Campos, F., Celi, S., Hess, B., and Kannwischer, M. J., "MAYO Specification Document, Round 2 Version," NIST PQC Additional Signatures, 2025. <https://csrc.nist.gov/csrc/media/Projects/pqc-dig-sig/documents/round-2/spec-files/mayo-spec-round2-web.pdf>
- [5]Beullens, W., "MAYO: Practical Post-Quantum Signatures from Oil-and-Vinegar Maps," *Selected Areas in Cryptography (SAC)*, 2022.
- [6]Aguilar Melchor, C., Aragon, N., Bettaieb, S., et al., "HQC: Hamming Quasi-Cyclic," NIST PQC, 2023. Selected as fifth PQC standard, March 2025 [13].
- [7]Bernstein, D. J., Chuengsatiansup, C., Lange, T., and van Vredendaal, C., "NTRU Prime: Reducing Attack Surface at Low Cost," *Selected Areas in Cryptography (SAC)*, 2017. IACR ePrint 2016/461. <https://eprint.iacr.org/2016/461>
- [8]National Institute of Standards and Technology, "Module-Lattice-Based Key-Encapsulation Mechanism Standard," FIPS 203, August 2024. <https://doi.org/10.6028/NIST.FIPS.203>
- [9]Botros, L., Kannwischer, M. J., and Schwabe, P., "Memory-efficient high-speed implementation of Kyber on Cortex-M4," *Africacrypt*, 2019.
- [10]Becker, H., Hwang, V., Kannwischer, M. J., Yang, B.-Y., and Yang, S.-Y., "Neon NTT: Faster Dilithium, Kyber, and Saber on Cortex-A72 and Apple M1," *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2022.
- [11]National Institute of Standards and Technology, "Post-Quantum Cryptography Standardization: Process, Status, and Next Steps," NISTIR 8413, 2022.
- [12]National Institute of Standards and Technology, "Status Report on the Second Round of the Additional Digital Signature Schemes for the NIST Post-Quantum Cryptography Standardization Process," NIST IR 8610, May 2026. <https://doi.org/10.6028/NIST.IR.8610>
- [13]National Institute of Standards and Technology, "NIST Selects HQC as Fifth Algorithm for Post-Quantum Encryption," March 2025. <https://www>.

- nist.gov/news-events/news/2025/03/
nist-selects-hqc-fifth-algorithm-post-quantum-encryption
- [14] Nejatollahi, H., Dutt, N., Ray, S., Regazzoni, F., Banerjee, I., and Cammarota, R., "Post-Quantum Lattice-Based Cryptography Implementations: A Survey," *ACM Computing Surveys*, vol. 51, no. 6, 2019.
 - [15] Longa, P. and Naehrig, M., "Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography," *CANS*, 2016.
 - [16] Kipnis, A., Patarin, J., and Goubin, L., "Unbalanced Oil and Vinegar Signature Schemes," *EUROCRYPT*, 1999.
 - [17] Arm Ltd., "Arm Architecture Reference Manual: Armv8, for A-profile architecture – NEON and Cryptographic Extensions," 2021.
 - [18] D'Anvers, J.-P., Karmakar, A., Roy, S. S., and Vercauteren, F., "Saber: Module-LWR Based Key Exchange, CPA-secure Encryption and CCA-secure KEM," *Africacrypt*, 2018.
 - [19] Bos, J., et al., "FrodoKEM: Learning With Errors Key Encapsulation," NIST PQC Round 3 Submission, 2020.
 - [20] Chung, C.-M. M., Hwang, V., Kannwischer, M. J., Seiler, G., Shih, C.-J., and Yang, B.-Y., "NTT Multiplication for NTT-unfriendly Rings: New Speed Records for Saber and NTRU on Cortex-M4 and AVX2," *IACR Transactions on Cryptographic Hardware and Embedded Systems (TCHES)*, 2021. IACR ePrint 2020/1397. <https://eprint.iacr.org/2020/1397>
 - [21] Harvey, D. and van der Hoeven, J., "On the Complexity of Integer Matrix Multiplication," *Foundations of Computational Mathematics*, 2014. (See also: Karatsuba, A. and Ofman, Y., "Multiplication of Multidigit Numbers on Automata," *Soviet Physics Doklady*, 1963.)
 - [22] Toeplitz, O., "Zur Theorie der quadratischen und bilinearen Formen von unendlichvielen Veränderlichen," *Mathematische Annalen*, vol. 70, 1911.